

■ NetPulse Analytics

Telecom Network Intelligence Dashboard
Project Overview & Teammate Contribution Guide

GitHub Repository

<https://github.com/3morad/Sla-breach>

Branch: **claude/xgboost-sla-streamlit-app-h10XP**

Main entry point: **app.py**

April 2026 · CESNET-TimeSeries24 · Streamlit Cloud

What Was Built

A **multi-model Streamlit dashboard** for Telecom ISPs that allows network engineers to upload hourly subnet traffic CSVs *once* and instantly analyse them across five AI/ML models — all from a single browser window, no coding required.

Project File Structure

File / Folder	Purpose
app.py	Home page — hero banner, 5 model cards, how-to guide
pages/upload.py	CSV upload — parses file, stores in shared session state
pages/member1_model.py	■ SLA Violation Risk — XGBoost inference (pre-trained)
pages/member2_model.py	■ Time-Series Forecasting — Holt-Winters ETS
pages/member3_model.py	■ Anomaly Detection — Isolation Forest (on-the-fly)
pages/member4_model.py	■ Clustering & Root Cause — K-Means + DBSCAN
pages/member5_model.py	■ Correlation & Dependency — Pearson/Spearman + network graph
utils/styles.py	Shared CSS, inject_css(), page_header() helpers
utils/feature_engineering.py	parse_csv, engineer_features, prepare_for_model
utils/api_client.py	call_endpoint() — FastAPI-ready HTTP client stub
sla_xgboost_model.json	Pre-trained XGBoost model weights (replace with real file)
model_config.pkl	Scaler + feature_cols + optimal_threshold (replace with real)
requirements.txt	All Python dependencies
.streamlit/config.toml	Dark theme + 200 MB upload limit

How the App Works

1. User goes to **Upload Data** in the sidebar and uploads a CSV once.
2. The CSV is parsed, datetime-indexed, and stored in `st.session_state["df"]` — shared automatically across every model page.
3. User navigates to any of the 5 model pages. Each page reads the shared data, runs its own analysis, and shows interactive Plotly charts.
4. Every page has a **Download CSV** button to export results.

The Five AI Models

■ Model 01 — Time-Series Forecasting

File	pages/member2_model.py
Covers (BOs)	BO1, BO3, BO7
Purpose	Predict future traffic KPIs (n_bytes, n_flows, etc.)
Algorithm	Holt-Winters Exponential Smoothing (ETS) with additive/multiplicative trend and seasonality
Output	Hourly forecast with 95% confidence intervals + STL decomposition (trend, seasonal, residual)
Sidebar controls	Forecast horizon (1–168 h), seasonal period (24 h / 168 h), trend & seasonality type
What teammate needs to provide	No pre-trained file needed — fits on uploaded data automatically

■ Model 02 — Anomaly Detection

File	pages/member3_model.py
Covers (BOs)	BO1, BO4, BO7
Purpose	Detect abnormal network behaviour without any labelled data
Algorithm	Isolation Forest (sklearn) trained in-browser on uploaded data
Output	Normalised anomaly score per hour, flagged events table, PCA scatter of normal vs anomaly
Sidebar controls	Contamination fraction (0.01–0.20), number of trees, rolling feature toggle
What teammate needs to provide	No pre-trained file needed — fits on uploaded data automatically

■ Model 03 — SLA Violation Risk

File	pages/member1_model.py
Covers (BOs)	BO1, BO4
Purpose	Predict the probability of an SLA breach for each hour
Algorithm	Pre-trained XGBoost binary classifier with optimal decision threshold
Output	Breach probability (0–1), risk timeline, breach-hours table, daily breach bar chart
Sidebar controls	Reads threshold from model_config.pkl — no manual tuning needed
What teammate needs to provide	■■ Requires: sla_xgboost_model.json + model_config.pkl in repo root

■ Model 04 — Clustering & Root Cause

File	pages/member4_model.py
Covers (BOs)	BO2, BO4, BO6
Purpose	Segment traffic hours into behavioural groups; localise fault patterns
Algorithm	K-Means or DBSCAN on StandardScaler + rolling-stats features, visualised with PCA 2-D
Output	Cluster label per hour (auto-named: Normal / Congested / Spiky / Low-traffic), cluster profiles table
Sidebar controls	Algorithm choice, k (K-Means) or eps + min_samples (DBSCAN)
What teammate needs to provide	No pre-trained file needed — fits on uploaded data automatically

■ Model 05 — Correlation & Dependency

File	pages/member5_model.py
Covers (BOs)	BO2
Purpose	Understand which KPIs move together; identify root-cause relationships
Algorithm	Pearson / Spearman static correlation, rolling windowed correlation, NetworkX dependency graph
Output	Heatmap, top-pairs bar chart, rolling correlation timeline, interactive network graph
Sidebar controls	Correlation method, rolling window (6–168 h), network edge threshold
What teammate needs to provide	No pre-trained file needed — computed directly from uploaded data

What Each Teammate Needs to Provide

Each model page is already coded and working. What teammates need to do is provide their **trained model files** (if applicable) and test with real data. Models that train on-the-fly need nothing extra.

Teammate	Model	Files to provide	Where to drop them
Member 1 (your model)	SLA Violation Risk (XGBoost)	sla_xgboost_model.json model_config.pkl model_config.pkl must contain: • scaler (StandardScaler) • feature_cols (list) • optimal_threshold (float)	Repo root /
Member 2	Time-Series Forecasting	No model file needed. App fits Holt-Winters on the uploaded CSV.	—
Member 3	Anomaly Detection	No model file needed. App trains Isolation Forest on the uploaded CSV.	—
Member 4	Clustering & Root Cause	No model file needed. App runs K-Means / DBSCAN on the uploaded CSV.	—
Member 5	Correlation & Dependency	No model file needed. All stats computed live from uploaded CSV.	—

CSV Format Expected

Every model page expects the same CSV format:

Requirement	Detail
Datetime column	Any column with timestamps (auto-detected). Name it 'timestamp', 'time', 'datetime', etc.
Numeric columns	One or more traffic metric columns: n_bytes, n_flows, packets, bytes_in, bytes_out, etc.
Frequency	Hourly resolution is required for rolling-24h and lag features to make sense.
Minimum rows	≥ 48 rows for forecasting; ≥ 25 rows for anomaly/clustering; any for correlation.
Missing values	Allowed — auto-imputed with column median before model inference.

Example first few lines of a valid CSV:

```
timestamp,bytes_in,bytes_out,packets
2024-01-01 00:00:00,1048576,819200,4800
2024-01-01 01:00:00,983040,753664,4512
2024-01-01 02:00:00,1126400,880640,5120
...
```

Feature Engineering (SLA model — auto-applied)

The SLA page automatically engineers these features before running XGBoost inference:

Feature group	What it creates	Example
Z-scores	Global z-score per column	{col}_zscore
Rolling 24h	mean, std, min, max over last 24 hours	{col}_roll24_mean, _std, _min, _max
Lag features	Values 1 hour ago, 6 hours ago, 24 hours ago	{col}_lag1, _lag6, _lag24
Time features	Hour of day, day of week, weekend flag, holiday	hour, dayofweek, is_weekend, is_holiday

Deployment & Quick-Start

Deploy to Streamlit Cloud

1. Go to **share.streamlit.io** and sign in with GitHub.
2. Click **New app** and fill in:
Repository : 3morad/Sla-breach
Branch : claude/xgboost-sla-streamlit-app-h10XP
Main file : app.py
3. Click **Deploy** — Streamlit Cloud installs requirements.txt automatically and gives you a public URL within ~2 minutes.
4. To update: push changes to the branch → Streamlit Cloud auto-redeploys.

Run Locally

```
git clone https://github.com/3morad/Sla-breach
cd Sla-breach
git checkout claude/xgboost-sla-streamlit-app-h10XP
pip install -r requirements.txt
streamlit run app.py
```

Add Your Real Model Files (Member 1 / SLA)

Replace the placeholder model files with your real trained files, then push:

```
# Drop your files into the repo root, then:
git add sla_xgboost_model.json model_config.pkl
git commit -m "Replace placeholder model with production artefacts"
git push
```

Streamlit Cloud detects the push and redeploys automatically.

Key Links

Resource	URL
GitHub Repo	https://github.com/3morad/Sla-breach
Branch	claude/xgboost-sla-streamlit-app-h10XP
Streamlit Cloud	https://share.streamlit.io
CESNET Dataset	https://zenodo.org/records/10958829
XGBoost docs	https://xgboost.readthedocs.io

Resource	URL
Streamlit docs	https://docs.streamlit.io